

Exploring Agentic AI in Anti-Forensics: Simulation of Evasion Tactics in Digital Investigations

1st Ifeoluwa Ojuolape-Oria

School of Science, Engineering and Environment
University of Salford
Manchester, UK
I.H.Ojuolape-Oria@edu.salford.ac.uk

2nd Oseodion Ofeimun

School of Science, Engineering and Environment
University of Salford
Manchester, UK
O.A.Ofeimun@edu.salford.ac.uk

3rd Emmanuel Orji

School of Science, Engineering and Environment
University of Salford
Manchester, UK
E.C.Orji@edu.salford.ac.uk

4th Olayinka Adeboye

School of Science, Engineering and Environment
University of Salford
Manchester, UK
A.A.Adeboye@salford.ac.uk

5th Aderemi Adenihun

School of Science, Engineering and Environment
University of Salford
Manchester, UK
A.Adenihun@edu.salford.ac.uk

6th Segun Akinseye

School of Science, Engineering and Environment
University of Salford
Manchester, UK
S.O.Akinseye@edu.salford.ac.uk

Abstract—Digital forensic practice is grounded in Locard’s Exchange Principle, which is based on the idea that malicious activity leaves detectable artefacts. Enterprise security architectures and forensic tools are therefore designed to recover traces generated by actions such as file modification, deletion, or configuration changes. However, the emergence of agentic artificial intelligence challenges this model by enabling autonomous systems to observe their environment, make decisions, and adapt their behaviour in real time to evade detection. This study empirically evaluates the anti-forensic capabilities of an autonomous agent, Anti-Gravity, developed to execute a multi-stage anti-forensic kill chain. The agent performs timestomping through metadata manipulation, secure wiping to permanently erase forensic artefacts, and log sanitisation to remove execution traces. Evaluation is conducted within a simulated enterprise environment monitored by a Security Information and Event Management (SIEM) platform incorporating File Integrity Monitoring (FIM). Results demonstrate a structural vulnerability in periodically polled monitoring systems, identified as a Time-of-Check to Time-of-Use (TOC-TOU) “Time Gap.” The agent completes its operations in under 300 milliseconds, entirely between scheduled FIM scans, resulting in no alerts or recorded artefacts within the SIEM. In contrast, kernel-level auditing captures low-level system calls associated with the attack, enabling partial forensic reconstruction.

Index Terms—Agentic Artificial Intelligence, Anti-Forensics, Digital Forensics, File Integrity Monitoring

I. INTRODUCTION

Digital forensic investigations rely on the availability and integrity of system artefacts to reconstruct past events. Artefacts such as log files, file system metadata, Master File Table

(MFT) entries, inode attributes, and process execution chains form the basis of forensic analysis and enable investigators to construct timelines and determine attribution [1]. The reliability of these artefacts is central to digital forensics, as the absence or manipulation of such evidence renders meaningful investigation impossible, a principle historically aligned with the foundational assumptions of forensic science [2]. Traditional anti-forensic activity has predominantly relied on manual techniques executed by human operators. Attackers have historically used command-line utilities such as `shred`, `touch`, or `sdelete` to conceal evidence when monitoring was suspected. These manual approaches are inherently constrained by human speed and error. Mistakes such as incomplete clean-up, typographical errors, or delays in execution frequently result in residual artefacts that can be captured by forensic tools or periodic monitoring mechanisms [1], [2]. The cybersecurity landscape has evolved with the emergence of agentic artificial intelligence. But right now, the cybersecurity landscape is changing in a big way because Agentic AI is being used as a weapon [3]. Agentic AI systems are characterised by autonomous decision-making, environmental awareness, and the ability to pursue complex, multi-step objectives without continuous human oversight [4]. Unlike traditional automation scripts that follow static execution paths, agentic systems maintain an internal representation of system state and adapt their actions dynamically. Recent research has demonstrated the use of agentic frameworks for offensive security operations, including vulnerability identification and automated exploitation [5], yet comparatively little attention has been

given to the post-exploitation and clean-up phases of attacks. This paper presents an Anti-Gravity autonomous agent specifically designed to execute a multi-stage anti-forensic kill chain. The kill chain consists of three anti-forensic techniques: timestomping through metadata manipulation, secure wiping to permanently erase forensic artefacts, and log sanitisation to remove traces of execution.

Despite advances in offensive techniques, many enterprise security architectures continue to rely on legacy monitoring paradigms. Security Information and Event Management (SIEM) platforms commonly implement File Integrity Monitoring through periodic polling to conserve system resources [6]. Under this model, this periodic polling approach is commonly retained to conserve system resources [6]. Under default enterprise scan intervals, malicious activity that occurs and is fully remediated between scans may remain invisible to the monitoring system, creating a Time-of-Check to Time-of-Use (TOCTOU) vulnerability [6]. File system states are examined at fixed intervals—often every 6, 12, or 24 hours—and compared against historical baselines. This design introduces a Time-of-Check to Time-of-Use (TOCTOU) vulnerability, where malicious activity that occurs and is fully remediated between scans may remain invisible to the monitoring system. Therefore, this paper empirically evaluates whether an agentic anti-forensic attack can evade industry-standard enterprise monitoring tools and assesses which layers of the operating system detection stack remain effective against high-speed autonomous evasion.

[6], [7]. This paper is motivated by the hypothesis that agentic AI possesses the speed and logical flexibility required to consistently exploit this temporal detection gap. By operating at machine speed, an autonomous agent may be able to create, manipulate, and remove forensic artefacts entirely within the interval between monitoring scans. The paper contributes by developing an AI anti-forensic tool that evades industry-standard enterprise monitoring tools and assesses which layers of the operating system detection stack remain effective against high-speed, autonomous evasion.

A. Contribution to the Study

This study presents an empirical evaluation of agentic AI-driven anti-forensics through the design and deployment of the Anti-Gravity autonomous agent. The project demonstrates that the agent can successfully execute timestomping, secure wiping, and log sanitisation entirely between periodic File Integrity Monitoring scans, thereby completely evading SIEM-based detection. In contrast, kernel-level auditing captures low-level system activity associated with the attack, providing forensic visibility at lower layers of the operating system.

II. LITERATURE REVIEW

To accurately situate this research within existing work, it is necessary to draw from three intersecting domains: agentic artificial intelligence, periodic versus continuous monitoring in digital forensics, and advanced anti-forensic techniques.

These areas collectively inform the assumptions, limitations, and experimental focus of this study.

A. Agentic Artificial Intelligence in Cybersecurity

The concept of agentic artificial intelligence has evolved from a theoretical cognitive model into a practical framework applicable to cybersecurity operations. Agentic AI systems are characterised as goal-directed entities capable of observing their environment, maintaining an internal state, making decisions, and executing actions autonomously without continuous human oversight [4]. Unlike traditional automation scripts that follow fixed, linear execution paths, agentic systems can adapt dynamically when confronted with obstacles or environmental changes. Recent research has demonstrated the offensive capabilities of agentic frameworks in cybersecurity contexts. Systems such as RedTeamLLM integrate large language models into autonomous agents capable of identifying vulnerabilities, generating exploit code, and executing penetration testing workflows [5]. Agentic AI has also been demonstrated in offensive contexts involving automated multi-stage attack execution [8]. However, the existing literature predominantly focuses on the intrusion and exploitation phases of the attack lifecycle. The post-exploitation phase, particularly clean-up and anti-forensic activity, remains underexplored. This gap is significant, as advanced threats place equal importance on evasion and attribution avoidance as they do on initial compromise.

B. Periodic Auditing and the Polling Problem

A longstanding challenge in digital forensics and security monitoring is the trade-off between system performance and detection fidelity. Traditional File Integrity Monitoring solutions, including those implemented in enterprise SIEM platforms such as Wazuh, rely on a periodic auditing model [6]. Under this approach, file systems are scanned at predefined intervals to establish baselines and detect deviations. This architecture introduces a well-known Time-of-Check to Time-of-Use (TOCTOU) vulnerability. If a file is created, modified, and removed entirely between scan intervals, the monitoring system may never observe its existence [6]. Enterprise SIEM platforms commonly implement File Integrity Monitoring using a periodic auditing model in which file system states are examined at fixed intervals and compared against historical baselines [6]. This approach introduces a Time-of-Check to Time-of-Use (TOCTOU) vulnerability: malicious activity that occurs and is fully remediated between scans may remain invisible to the monitoring system [6]. While continuous or event-driven monitoring models offer improved detection accuracy, they impose significant computational and storage overheads, leading many organisations to retain periodic auditing configurations [9]. This reliance on periodic scanning creates a temporal blind spot that may be exploitable by high-speed adversaries. The literature recognises this limitation in theory; however, empirical evaluation of this weakness in the presence of autonomous, machine-speed attackers remains

limited. This study directly evaluates the implications of this design choice under controlled experimental conditions.

C. Forensic Artefacts and File System Evidence

Digital forensic investigations rely on the availability and integrity of system artefacts to reconstruct past events. Artefacts such as log files, file system metadata, Master File Table (MFT) entries, inode attributes, and process execution chains form the basis of forensic analysis and enable investigators to construct timelines and determine attribution. The reliability of these artefacts is central to digital forensics, as the absence or manipulation of such evidence renders meaningful investigation impossible [2].

D. Advanced Anti-Forensic Techniques

Anti-forensics encompasses techniques intended to destroy, conceal, or manipulate digital evidence to undermine forensic investigation. Garfinkel categorised these techniques as evidence destruction, evidence hiding, and elimination of evidence sources [2]. Modern anti-forensic activity extends beyond simple file deletion to target file system metadata and logging mechanisms. The anti-forensic techniques evaluated in this study target different categories of forensic evidence. Timestomping is performed through metadata manipulation, which affects timestamp information relied upon during forensic analysis. Secure wiping is used to permanently erase forensic artefacts by overwriting file contents prior to deletion, preventing recovery. Log sanitisation is performed to remove traces of execution by removing evidence recorded in logs.

One widely documented technique is timestomping, which involves manipulating file timestamps stored in metadata structures such as the Master File Table or inode attributes. While tools exist to perform timestomping manually, such actions often introduce detectable anomalies between timestamp fields that can be identified during forensic analysis [1]. Similarly, standard file deletion does not remove underlying data; instead, it marks storage blocks as unallocated, allowing forensic recovery unless deliberate overwriting is performed. Secure wiping addresses this limitation by overwriting file contents prior to deletion, rendering recovery infeasible. However, manual execution of secure wiping operations is time-consuming and generates additional system activity that may increase the likelihood of detection. The literature, therefore, suggests that while anti-forensic techniques are effective in principle, their success is often constrained by execution speed and operational noise.

E. Manual Versus Agentic Anti-Forensics

This section highlights key operational differences between manual anti-forensic activity and agentic execution. Manual attacks are limited by human speed, static decision-making, and a higher likelihood of error. In contrast, agentic AI systems execute actions at machine speed, verify task completion autonomously, and adapt execution logic as needed [4], [5]. This distinction has direct forensic implications. Manual attacks are more likely to leave residual artefacts detectable by

Feature	Human/Manual Attack	Agentic AI Attack (Our Project)
Execution Speed	Slow (Seconds to Minutes)	Instant (Milliseconds)
Decision Making	Static (Follows a playbook)	Adaptive (Verifies success)
Forensic Footprint	High (Human error is likely)	Low (Perfect execution)
Detection Risk	Caught by standard scans	Evades Periodic Scans

Fig. 1. Comparison of Manual and Agentic Anti-Forensic Characteristics

periodic monitoring, whereas agentic systems may complete entire anti-forensic workflows within sub-second timeframes (as shown in Figure 1). While this difference is recognised conceptually in the literature, empirical validation against enterprise monitoring systems remains limited. This paper builds directly on these documented distinctions by experimentally evaluating agentic anti-forensics within a realistic monitoring environment.

F. Summary

The reviewed literature establishes that agentic AI introduces a fundamental shift in attack execution speed and autonomy, while enterprise monitoring systems continue to rely heavily on periodic auditing mechanisms. Advanced anti-forensic techniques are well documented individually; however, their automated, agentic execution remains insufficiently examined. This paper empirically evaluates agentic anti-forensics against industry-standard monitoring configurations, as derived directly from the project implementation and results.

III. METHODOLOGY

This paper adopts a planned, staged experimental methodology to evaluate the effectiveness of agentic artificial intelligence-driven anti-forensic activity against enterprise monitoring systems. The methodology progresses from a preliminary baseline experiment to a fully autonomous anti-forensic simulation, allowing the impact of agentic behaviour to be isolated and evaluated under controlled conditions.

A. Experimental Approach

The methodology follows a two-phase structure. The first phase establishes a baseline using a rule-based agent to observe standard detection behaviour. The second phase introduces a fully autonomous agent designed to execute anti-forensic actions at machine speed. This staged approach ensures that observed evasion outcomes can be attributed specifically to agentic execution rather than to environmental misconfiguration or tool failure.

B. Phase 1: Preliminary Rule-Based Baseline

The preliminary phase is conducted to establish baseline detection behaviour and identify limitations in rule-based automation. This experiment is performed on a Windows 11 target system accessed from a Kali Linux attacker node. A rule-based agent is implemented using static heuristics and detection feedback. The agent monitors Sysmon Event ID 1 (Process Creation) and attempts to halt execution when known monitoring processes are detected. While this approach

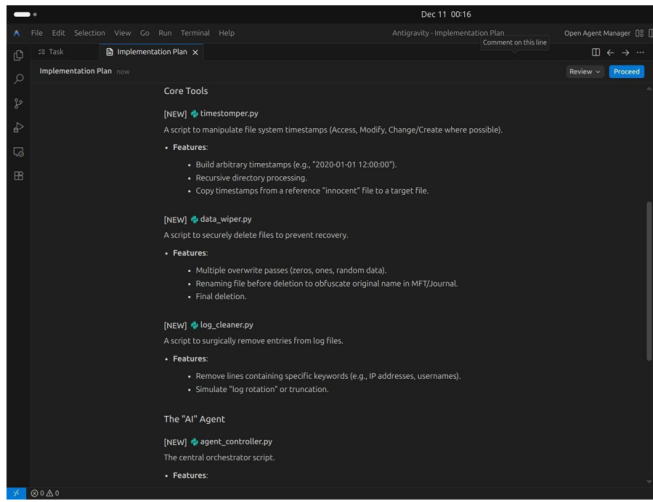


Fig. 2. Anti-Gravity modular architecture/implementation plan

provides initial insight into detection patterns, it is limited by fixed logic and constrained by slow execution speed. The agent is unable to adapt to different logging configurations or operate fast enough to evade periodic monitoring consistently, motivating the design of a fully autonomous solution. The preliminary rule-based baseline highlights the limitations of static anti-detection logic and motivates the development of a fully autonomous agent capable of executing a multi-stage anti-forensic operation without continuous human oversight. This directly informs the design and evaluation of the Anti-Gravity autonomous agent in the second phase of the methodology. Phase 1 establishes a baseline rule-based detection approach using Sysmon process creation logging (Event ID 1) on Windows 11. Phase 2 then empirically evaluates the Anti-Gravity autonomous agent within a simulated enterprise environment monitored by an industry-standard SIEM stack incorporating File Integrity Monitoring and endpoint telemetry.

C. Phase 2: The Anti-Gravity Autonomous Agent

The second phase introduces Anti-Gravity, a modular, Python-based autonomous agent developed for Linux systems. The agent is designed to operate without continuous human oversight and to execute anti-forensic actions as a single, rapid operation.

D. Agent Architecture

The agent is implemented using a modular architecture controlled by a central logic engine (`agent_controller.py`) (as shown in Figure 2). Individual anti-forensic capabilities are encapsulated within separate modules, allowing the agent to chain techniques dynamically without modifying the core control logic.

E. Time Manipulation Engine

The timestomping module (`timestomper.py`) manipulates file metadata by directly invoking the operating system file system

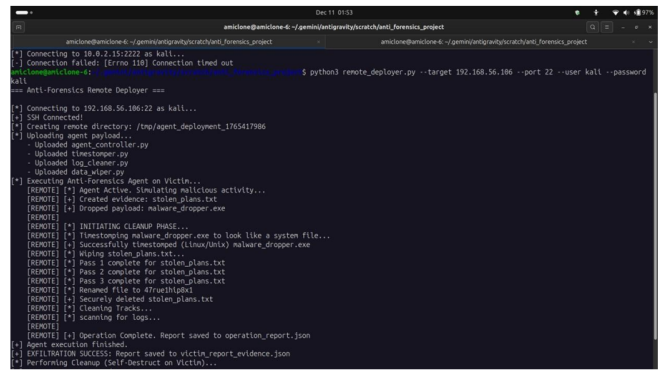


Fig. 3. Remote deployment and attack launch

API. Instead of using external utilities such as the `touch` command, which generate additional process creation artefacts, the agent modifies timestamps programmatically using Python's `os.utime()` function. This approach avoids spawning new shell processes and reduces observable execution artefacts.

F. Secure Wiping Module

The secure wiping module (`data_wiper.py`) implements a three-stage wiping process. First, file contents are overwritten with random data to destroy file headers. Second, filenames are randomised to alter metadata associations. Finally, files are unlinked from the file system. This sequence ensures that deleted data is rendered unrecoverable and that residual artefacts are minimised.

G. Operational Workflow

The experiment follows a fixed execution sequence to ensure reproducibility:

- **Baseline Creation:** The target system is populated with simulated sensitive data and verified prior to execution.
- **Injection:** The agent is deployed remotely using an encrypted SSH connection established by the `remote_deployer.py` script. The Anti-Gravity agent autonomously executes a multi-stage anti-forensic kill chain consisting of timestomping through metadata manipulation, secure wiping to permanently erase forensic artefacts, and log sanitisation to remove traces of execution. The agent operates entirely between scheduled FIM scans configured at default intervals.
- **Execution:** The agent payload is uploaded to a temporary directory and executed in stealth mode to initiate the anti-forensic kill chain.
- **Verification:** Following execution, the file system state and monitoring logs are examined to assess detection outcomes (as shown in Figure 3).

H. Monitoring Configuration

To evaluate detection capability at different system layers, a defence-in-depth monitoring stack is implemented. User-mode monitoring is provided by a SIEM-based File Integrity Monitoring system configured with periodic polling at default

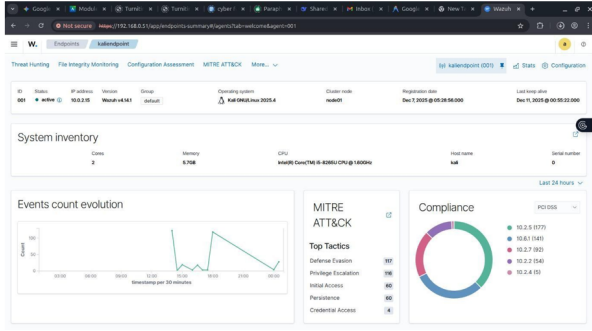


Fig. 4. Wazuh agent status and FIM configuration

intervals (as shown in Figure 4). Real-time monitoring is intentionally disabled to reflect performance-conscious enterprise deployments. Kernel-level monitoring is implemented using the Linux Audit Daemon (auditd). Audit rules are configured to monitor write and attribute-change operations within the target directory, enabling capture of low-level system calls independent of polling intervals.

The monitoring configuration is intentionally aligned with the periodic auditing model commonly deployed in enterprise environments. File Integrity Monitoring is configured to run at default scan intervals, creating a Time-of-Check to Time-of-Use (TOCTOU) window in which file system activity may occur and be remediated between scans. To evaluate the TOCTOU “Time Gap” introduced by periodic auditing, the Anti-Gravity agent execution is assessed relative to the configured File Integrity Monitoring scan schedule. The autonomous agent is designed to complete the full anti-forensic operation in under 300 milliseconds, operating entirely between scheduled FIM scans configured at default intervals. The presence of the Time-Gap is confirmed by comparing monitoring outputs: no file modification, deletion, or timestamp alteration events are recorded by the SIEM during execution, while kernel-level auditing captures low-level system calls associated with the operation. The study evaluates whether the autonomous agent can complete timestamping, secure wiping, and log sanitisation entirely within this interval. This directly tests the presence of a structural “Time Gap” in periodically polled monitoring systems, where activity executed between scheduled scans may not be recorded by the SIEM.

I. Data Collection and Evaluation

Following agent execution, monitoring outputs are collected from both the SIEM dashboard and kernel audit logs. The SIEM is examined for alerts related to file deletion, modification, or timestamp manipulation. Kernel audit logs are queried to identify system calls associated with the agent’s actions. Detection effectiveness is assessed by comparing the presence or absence of forensic artefacts across user-mode and kernel-mode monitoring layers.

J. Evaluation Criteria

Detection effectiveness is evaluated by comparing monitoring outputs from the SIEM File Integrity Monitoring layer

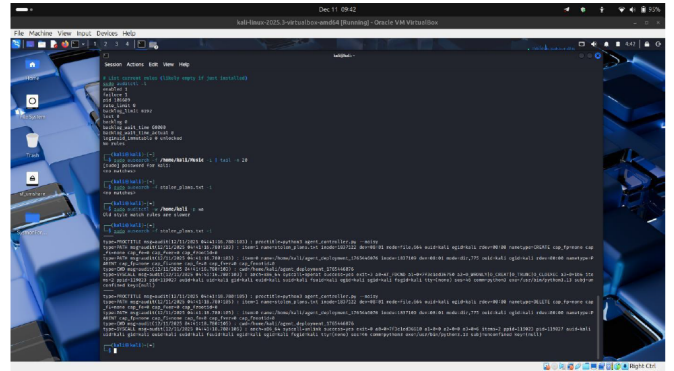


Fig. 5. Kernel audit log evidence of system calls

and the kernel-level audit layer (as shown in Figure 5). The SIEM output is assessed for recorded events indicating file modification, deletion, or timestamp alteration within the monitored directory. The kernel-level audit output is assessed for low-level system call evidence associated with the anti-forensic operation.

Successful evasion at the SIEM layer is defined as completion of the full anti-forensic operation without file modification, deletion, or timestamp alteration events being recorded by the SIEM. Kernel-level detection is defined as the presence of audit log evidence enabling partial forensic reconstruction through recorded system calls.

IV. IMPLEMENTATION

This section describes the implementation of the autonomous anti-forensic agent and the monitoring infrastructure used in this paper. All components are deployed within a controlled virtualised environment to ensure repeatable execution and consistent observation.

A. Experimental Environment and Infrastructure

The experimental environment is implemented using Oracle VirtualBox to construct a controlled cyber-range simulating a corporate sub-network subject to lateral movement and post-exploitation activity. A three-node topology is deployed to separate attacker infrastructure, the victim endpoint, and centralised monitoring. The attacker node operates as a command-and-control (C2) server running Ubuntu 22.04 LTS and hosts the `remote_deployer.py` script responsible for payload delivery and execution orchestration. As shown in Figure 3, the `remote_deployer.py` script is executed from the attacker node to deliver the payload and orchestrate remote execution on the victim endpoint. The victim node runs Kali Linux 2024 and simulates a compromised high-value workstation containing sensitive data represented by dummy files stored in `/home/kali/Music`. Monitoring is provided by a dedicated Wazuh SIEM virtual appliance configured to aggregate logs from the victim endpoint via the Wazuh agent [6]. To enable realistic network interaction, a bridged adapter configuration is used. This assigns the victim system an IP address on the same physical subnet as the attacker, enabling inbound

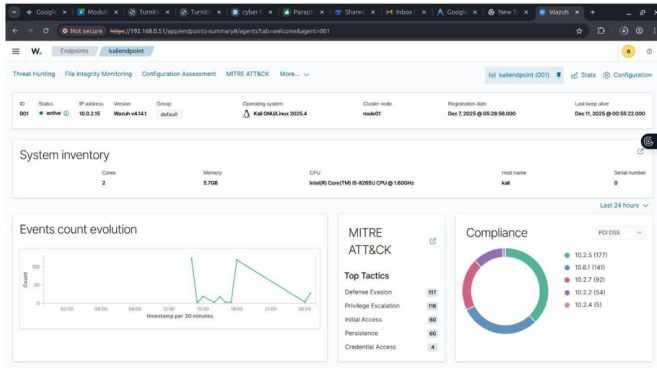


Fig. 6. Wazuh agent status and FIM configuration

SSH connections and simulating lateral movement within a local area network. This configuration mirrors real-world post-compromise scenarios and is required to support remote agent deployment.

B. Defence-in-Depth Monitoring Stack

A layered monitoring approach is implemented to evaluate detection capability at different levels of the operating system abstraction. User-mode monitoring is provided by Wazuh File Integrity Monitoring (FIM) (as shown in Figure 6), implemented through the syscheck module. The configuration explicitly monitors the target directory `/home/kali/Music`, while retaining the default polling interval of twelve hours and disabling real-time monitoring. This configuration reflects performance-conscious enterprise deployments and intentionally exposes the system to the Time-of-Check to Time-of-Use (TOCTOU) vulnerability associated with periodic auditing [6]. Kernel-level monitoring is implemented using the Linux Audit Daemon (auditd). Audit rules are configured to monitor write and attribute-change operations within the target directory, capturing low-level system calls related to file modification and deletion. Unlike periodic FIM, auditd operates in an event-driven manner by intercepting system calls directly at the kernel level.

C. Autonomous Agent Architecture

The autonomous agent, named Anti-Gravity, is implemented as a modular Python-based offensive tool designed for Linux systems (as shown in Figure 7). The agent is controlled by a central orchestration script (`agent_controller.py`) that coordinates execution of individual anti-forensic modules. This modular design enables techniques to be chained dynamically without modification of the core logic. The agent is designed to operate autonomously once initiated and does not require continuous human interaction. All actions are executed within a single, rapid operation cycle to minimise observable artefacts and exploit the latency of periodic monitoring mechanisms.

D. Time Manipulation Engine

The timestomping capability is implemented in the `timestomper.py` module. Attackers often use the Linux `touch`

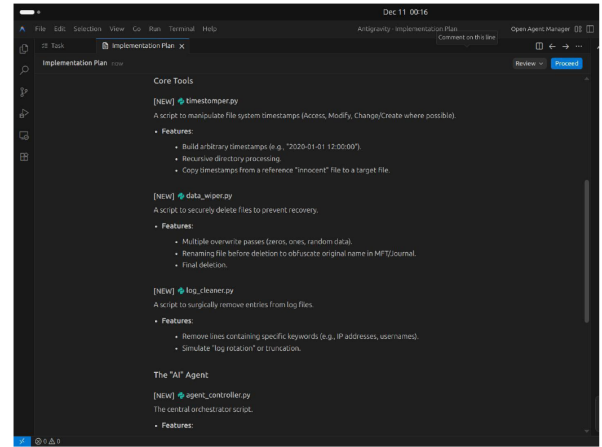


Fig. 7. Anti-Gravity modular implementation plan

Code Logic:

```
def stomp_file(filepath, target_date="2010-01-01 12:00:00"):
    # Convert string to epoch time
    epoch_time = time.mktime(datetime.strptime(target_date,
    ...).timetuple())
    # Apply to Access (atime) and Modification (mtime)
    os.utime(filepath, (epoch_time, epoch_time))
```

Fig. 8. Time stomper Code Logic

command to change the timestamps on files, a technique widely documented in forensic literature [10] (as shown in Figure 8). Rather than invoking external utilities such as the Linux `touch` command—which would generate additional process creation records—the agent directly modifies file timestamps using Python’s `os.utime()` system call interface. This approach avoids spawning new shell processes and reduces the forensic footprint associated with metadata manipulation.

Timestamps are programmatically backdated to values outside common “recent activity” windows used during forensic timeline analysis, thereby concealing evidence of recent file interaction.

E. Secure Wiping Module

The secure wiping functionality is implemented in the `data_wiper.py` module and follows a three-stage process. First, file contents are overwritten with random data generated using `os.urandom()` to destroy file headers and prevent signature-based recovery. Second, filenames are replaced with random alphanumeric strings to modify inode associations. Finally, files are unlinked from the file system, marking the associated storage blocks as free space. This sequence ensures that deleted data is rendered unrecoverable by forensic recovery tools while minimising residual metadata.

F. Log Sanitisation Module

Log sanitisation is implemented in the `log_cleaner.py` module as part of the Anti-Gravity multi-stage anti-forensic kill

```
> amiconline > gemini > antigravity > scratch > anti_forensics_project > agent_package > agent_controller.py
def run_cleanup():
    if mode == "stealth":
        else:
            try:
                report["actions"].append({"tool": "standard_delete", "target": EVIDENCE_FILE})
            except Exception as e:
                print(f"[!] Delete failed: {e}")

# 3. Clean Logs
if mode == "stealth":
    print(f"[!] Cleaning Tracks...")
    log_cleaner.clean_logs()
    report["actions"].append({"tool": "log_cleaner", "status": "attempted"})
else:
    print(f"[!] NOISY: Skipping Log Cleaning (Traces left intact)")
    report["actions"].append({"tool": "log_cleaner", "status": "skipped noisy"})

# Write report
with open(REPORT_FILE, "w") as f:
    json.dump(report, f, indent=4)
    print(f"\n[+] Operation Complete. Report saved to {REPORT_FILE}")

if __name__ == "__main__":
    generate_activity()
    run_cleanup()
```

Fig. 9. Log sanitisation

chain. The module is designed to remove traces of execution by surgically modifying log files to eliminate entries associated with the agent's activity. (as shown in Figure 9) The implementation supports targeted removal of lines containing specified keywords, including identifiers such as IP addresses and usernames. In addition, the module can simulate log rotation or truncation behaviour to reduce the persistence of relevant entries within monitored logs. Log sanitisation is implemented as part of the Anti-Gravity multi-stage anti-forensic kill chain. The capability is encapsulated as a dedicated module (`log_cleaner.py`) within the agent's modular architecture. This stage is designed to remove traces of execution following the completion of timestomping and secure wiping.

G. Operational Workflow

The execution of the agent follows a fixed workflow to ensure repeatability:

- **Baseline creation:** The victim system is populated with simulated sensitive data and verified prior to execution.
- **Injection:** The `remote_deployer.py` script establishes an encrypted SSH connection to the victim using the Paramiko library and uploads the agent payload to a temporary directory.
- **Execution:** The agent is executed in stealth mode, triggering the anti-forensic kill chain.
- **Verification:** Following execution, the file system state and monitoring outputs from Wazuh and auditd are examined. (as shown in Figure 10)

H. Evidence Collection

Following execution, detection outcomes are assessed by reviewing the Wazuh SIEM dashboard for file integrity alerts and by querying kernel audit logs using ausearch. Kernel-level

[illegible]

Fig. 10. Remote deployment and attack launch

[illegible]

Fig. 11. Kernel audit log evidence showing truncate and unlink system calls

logs are examined to reconstruct the sequence of system calls associated with timestamp modification, file truncation, and deletion (as shown in Figure 11).

V. SUMMARY OF DETECTION CAPABILITIES

Figure summarises the effectiveness of the deployed monitoring stack against the autonomous agent. Wazuh File Integrity Monitoring is configured with a default 12-hour scan frequency and realtime disabled, and under this periodic model the SIEM produces no file integrity alerts for timestomping, secure wiping, or log cleaning. In contrast, the kernel-mode layer (auditd) captures the underlying system call activity associated with each stage of the operation, including timestomping evidence through `syscall=utime`, secure wiping evidence through `O_TRUNC` followed by `unlink`, and log cleaning evidence through file access events corresponding to bash history clearing. These kernel events are correlated using a dedicated audit identifier (AuditID), implemented as a label applied within auditd rule configuration, which tags and groups all relevant system call records at the kernel level into a single investigative context. In this study, a specific AuditID label (`anti_forensics_watch`) was used to associate all anti-forensic activity within the monitored directory, enabling consistent identification of related kernel events across multiple attack phases. This AuditID allows investigators to directly query and reconstruct attack activity from kernel audit logs, even when files are created, modified, and deleted within sub-second

timeframes that fall entirely outside the visibility window of periodic File Integrity Monitoring.

Unlike Wazuh’s user-mode FIM, which relies on scheduled filesystem state comparisons and therefore suffers from Time-of-Check to Time-of-Use (TOCTOU) blind spots, auditd operates in an event-driven manner at the syscall interface, recording actions at the moment they occur. The presence of the AuditID ensures that these low-level events are not isolated log entries but form a coherent execution narrative spanning multiple anti-forensic stages. This distinction explains why Wazuh reports complete evasion across all attack phases in Figure 12, while auditd consistently detects each phase and enables partial forensic reconstruction despite the agent’s successful data sanitisation. Overall, the table shows that the agent evades periodic FIM at the SIEM layer while still leaving kernel-level traces enabling partial forensic reconstruction.

Attack Phase	Wazuh (Periodic FIM)	Auditd (Real-Time Kernel)	Forensic Outcome
Timestomping	FAILED (Missed changes)	DETECTED (syscall=utime)	Metadata successfully backdated.
Secure Wiping	FAILED (Time Gap)	DETECTED (O_TRUNC + Unlink)	Data rendered unrecoverable.
Log Cleaning	FAILED (No alert)	DETECTED (File Access)	Bash history cleared.
Overall Status	EVADED	DETECTED	Partial Forensic Recovery

Fig. 12. Summary of Detection Capabilities

VI. CONCLUSION AND FUTURE WORK

The experimental results demonstrate a structural vulnerability in periodically polled monitoring systems, identified as a Time-of-Check to Time-of-Use (TOCTOU) “Time Gap.” The Anti-Gravity autonomous agent completed the full anti-forensic operation in under 300 milliseconds, operating entirely between scheduled File Integrity Monitoring scans configured at default intervals. As a result, no file modification, deletion, or timestamp alteration events were recorded by the SIEM. In contrast, kernel-level auditing successfully captured low-level system calls associated with the attack, enabling partial forensic reconstruction.

The failure of periodic File Integrity Monitoring in this experiment can be explained through the theoretical lens of signal processing. The Nyquist–Shannon Sampling Theorem states that to perfectly reconstruct a signal, one must sample at a rate greater than twice the highest frequency of the signal [11]. In this context, periodically polled monitoring can be interpreted as sampling system state at discrete intervals. When anti-forensic activity occurs entirely between sampling points, the monitoring system may fail to observe the activity and therefore cannot reconstruct the event sequence.

This study empirically evaluated the anti-forensic capabilities of an autonomous agent, Anti-Gravity, developed to execute a multi-stage anti-forensic kill chain consisting of timestomping through metadata manipulation, secure wiping to permanently erase forensic artefacts, and log sanitisation to remove traces of execution. The evaluation was conducted within a simulated enterprise environment monitored by a SIEM stack incorporating File Integrity Monitoring (FIM).

These findings provide empirical evidence that periodic auditing mechanisms are insufficient against high-speed, autonomous anti-forensic activity and highlight the risks of relying on default enterprise monitoring configurations in the presence of agentic AI-driven threats.

Future work should further examine detection capability across different layers of the operating system detection stack in the presence of high-speed autonomous anti-forensic execution.

REFERENCES

- [1] H. Carvey, *Windows Forensic Analysis Toolkit*. Elsevier / Syngress, 4th ed., 2014. Updated to cover Windows 8 systems.
- [2] S. L. Garfinkel, “Anti-forensics and the digital investigator,” in *Proceedings of the 5th Australian Digital Forensics Conference*, (Perth, Western Australia), 2007.
- [3] A. Juneja, “Rx-int: A kernel engine for real-time detection and analysis of in-memory threats,” *arXiv preprint arXiv:2508.03879*, 2025.
- [4] D. B. Acharya, K. Kuppan, and B. Divya, “Agentic ai: Autonomous intelligence for complex goals—a comprehensive survey,” *IEEE Access*, 2025.
- [5] B. Challita and P. Parrend, “Redteamllm: an agentic ai framework for offensive security,” *arXiv preprint arXiv:2505.06913*, 2025.
- [6] Wazuh Documentation Team, “File integrity monitoring: How it works,” 2024. [Online]. Accessed: 2026-01-15.
- [7] J. Wei and C. Pu, “Toctou vulnerabilities in unix-style file systems: An anatomical study,” in *FAST*, vol. 5, pp. 12–12, 2005.
- [8] X. Li, Y. Chen, and M. Xu, “Automating multi-stage cyber attacks with artificial intelligence,” *IEEE Transactions on Information Forensics and Security*, vol. 18, pp. 992–1007, 2023.
- [9] M. Eulerich, N. Waddoups, M. Wagener, and D. A. Wood, “Development of a framework of key internal control and governance principles for robotic process automation (rpa),” *Journal of Information Systems*, vol. 38, no. 2, pp. 29–49, 2024.
- [10] A. Kroll, “Breaking time: Methods, artifacts, and forensic detection of timestomping on fat32, ext3, and ext4 file systems,” tech. rep., SANS Institute Information Security Reading Room, 2025. White paper.
- [11] C. E. Shannon, “Communication in the presence of noise,” *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949.